

Instruction Set

Use the following instruction set when writing assembly code (.asm) files:

Opcode	Ra	Rb	Rc	nn	Instruction	
00000	000	000	ccc	00	LDI RC,data	Load RC with data
00001	000	000	000	00	JMP addr	Jump to addr
00010	000	000	111	00	CALL addr	Call the subroutine at addr
00011	000	000	00000		JEQ addr	Jump to addr if equal
00011	000	000	00001		JNE addr	Jump to addr if not equal
00011	000	000	00010		JZ addr	Jump to addr if zero
00011	000	000	00011		JNZ addr	Jump to addr if not zero
00011	000	000	00100		JC addr	Jump to addr if carry
00011	000	000	00101		JNC addr	Jump to addr if no carry
00011	000	000	00110		JGT addr	Jump to addr if '>'
00011	000	000	00111		JGE addr	Jump to addr if '>='
00011	000	000	01000		JLT addr	Jump to addr if '<'
00011	000	000	01001		JLE addr	Jump to addr if '<='
00011	000	000	01010		JPL addr	Jump to addr if positive
00011	000	000	01011		JMI addr	Jump to addr if negative
00011	000	000	01100		JV addr	Jump to addr if overflow
00011	000	000	01101		JNV addr	Jump to addr if no overflow
00100	aaa	000	ccc	00	LD RC,[RA]	Load RC from addr in RA
00100	aaa	000	000	01	SETHP [RA]	Set Heap End in SP from addr in RA
00101	aaa	bbb	ccc	00	LDX RC,[RA,RB]	Load RC from addr in (RA + RB)
00110	aaa	000	ccc	00	ST RA,[RC]	Store RA to addr in RC
00111	aaa	bbb	ccc	00	STX RA,[RC,RB]	Store RA to addr in (RC + RB)
01000	aaa	000	ccc	00	MOV RA,RC	Copy RA to RC
01001	000	000	ccc	00	MOVSP RC	Copy SP to RC
01010	aaa	bbb	000	00	SETPX RA,RB	Set the pixel at position RA to colour RB
01011	aaa	bbb	ccc	00	FSETPX RA,RB,RC	Set the pixel at pos (RA, RB) to col RC
01100	000	000	000	00	DSI	Disable interrupts

01100	000	000	000	01	ENI	Enable interrupts
01101	000	000	000	00	RTI	Return from interrupt
01110	aaa	000	000	00	JMPR RA	Jump to addr in RA
01110	111	000	000	00	RET	Return from subroutine
01111	000	000	ccc	00	POP RC	Pop from the stack to RC
10000	aaa	000	000	00	PUSH RA	Push RA onto the stack
10001	aaa	bbb	ccc	00	ADD RA,RB,RC	$RC = RA + RB$
10010	aaa	bbb	ccc	00	LSL RA,RB,RC	$RC = RA \ll RB$
10011	aaa	bbb	ccc	00	LSR RA,RB,RC	$RC = RA \gg RB$
10011	aaa	bbb	ccc	01	ASR RA,RB,RC	$RC = RA \gg RB$ (maintaining the msb)
10100	aaa	000	000	00	INC RA	$RA = RA + 1$
10101	aaa	000	000	00	DEC RA	$RA = RA - 1$
10110	aaa	000	ccc	00	NOT RA,RC	$RC = !RA$
10111	aaa	bbb	ccc	00	AND RA,RB,RC	$RC = RA \& RB$
11000	aaa	bbb	ccc	00	OR RA,RB,RC	$RC = RA RB$
11001	aaa	bbb	ccc	00	XOR RA,RB,RC	$RC = RA \wedge RB$
11001	aaa	aaa	aaa	00	CLR RA	$RA = 0$
11010	aaa	bbb	000	00	CMP RA,RB	Compare RA against RB & set A,E flags
11010	aaa	bbb	000	01	FCMP RA,RB	Compare RA against RB & set A,E flags
11011	000	000	000	00	HALT	Halt the program
11100	aaa	bbb	ccc	00	SUB RA,RB,RC	$RC = RA - RB$
11100	aaa	bbb	ccc	01	MUL RA,RB,RC	$RC = RA * RB$
11100	aaa	bbb	ccc	10	SDIV RA,RB,RC	$RC = RA / RB$ (signed)
11100	aaa	bbb	ccc	11	UDIV RA,RB,RC	$RC = RA / RB$ (unsigned)
11101	aaa	bbb	ccc	00	FADD RA,RB,RC	$RC = RA + RB$
11101	aaa	bbb	ccc	01	FSUB RA,RB,RC	$RC = RA - RB$
11101	aaa	bbb	ccc	10	FMUL RA,RB,RC	$RC = RA * RB$
11101	aaa	bbb	ccc	11	FDIV RA,RB,RC	$RC = RA / RB$
11110	aaa	000	ccc	00	FSQRT RA,RC	$RC = \text{sqrt}(RA)$
11110	aaa	000	ccc	01	FSIN RA,RC	$RC = \sin(RA)$: RA in range [0,90]
11110	aaa	000	ccc	10	FCOS RA,RC	$RC = \cos(RA)$: RA in range [0,90]

11110	aaa	000	ccc	11	FTAN RA,RC	RC = tan(RA) : RA in range [0,90]
11111	000	000	000	00	END	Stop the program

Notes:

- 1) The instructions starting with an 'F' are floating point instructions.
- 2) The conditional jumps use signed numbers.